

A Technical Introduction to Mixture of experts

Section 1

For each input-output pair (x, y) , the weighting function is changed to increase the weight on all experts that performed above average, and decrease the weight on all experts that performed below average. This encourages the weighting function to learn to select only the experts that make the right predictions for each input. The i -th expert is changed to make its prediction closer to y , but the amount of change is proportional to $w(x)_i N(y|\mu_i, I)$. This has a Bayesian interpretation. Given input x , the prior probability that expert i is the right one is $w(x)_i$, and $N(y|\mu_i, I)$ is the likelihood of evidence y . So, $w(x)_i N(y|\mu_i, I) / \sum_j w(x)_j N(y|\mu_j, I)$ is the posterior probability for expert i , and so the rate of change for the i -th expert is proportional to its posterior probability. In words, the experts that, in hindsight, seemed like the good experts to consult, are asked to learn on the example. The experts that, in hindsight, were not, are left alone. The combined effect is that the experts become specialized: Suppose two experts are both good at predicting a certain kind of input, but one is slightly better, then the weighting function would eventually learn to favor the better one. After that happens, the lesser expert is unable to obtain a high gradient signal, and becomes even worse at predicting such kind of input. Conversely, the lesser expert can become better at predicting other kinds of input, and increasingly pulled away into another region. This has a positive feedback effect, causing each expert to move apart from the rest and take care of a local region alone (thus the name "local experts").

Section 2

Routing In the original sparsely-gated MoE, only the top- k experts are queried, and their outputs are weighted-summed. There are other methods. Generally speaking, routing is an assignment problem: How to assign tokens to experts, such that a variety of constraints are followed (such as throughput, load balancing, etc.)? There are typically three classes of routing algorithm: the experts choose the tokens ("expert choice"), the tokens choose the experts (the original sparsely-gated MoE), and a global assigner matching experts and tokens. During inference, the MoE works over a large batch of tokens at any time. If the tokens were to choose the experts, then some experts might get few tokens, while a few experts get so many tokens that it exceeds their maximum batch size, so they

would have to ignore some of the tokens. Similarly, if the experts were to choose the tokens, then some tokens might not be picked by any expert. This is the "token drop" problem. Dropping a token is not necessarily a serious problem, since in Transformers, due to residual connections, if a token is "dropped", it does not disappear. Instead, its vector representation simply passes through the feedforward layer without change. Other approaches include solving it as a constrained linear programming problem, using reinforcement learning to train the routing algorithm (since picking an expert is a discrete action, like in RL). The token-expert match may involve no learning ("static routing"): It can be done by a deterministic hash function or a random number generator.

Section 3

$\theta = (\theta_0, \theta_1, \dots, \theta_n)$ is the set of parameters. The parameter θ_0 is for the weighting function. The parameters $\theta_1, \dots, \theta_n$ are for the experts. Given an input x , the mixture of experts produces a single output by combining $f_1(x), \dots, f_n(x)$ according to the weights $w(x)_1, \dots, w(x)_n$ in some way, usually by $f(x) = \sum_i w(x)_i f_i(x)$. Both the experts and the weighting function are trained by minimizing some loss function, generally via gradient descent. There is much freedom in choosing the precise form of experts, the weighting function, and the loss function.

Section 4

Capacity factor Suppose there are n experts in a layer. For a given batch of queries $\{x_1, x_2, \dots, x_T\}$, each query is routed to one or more experts. For example, if each query is routed to one expert as in Switch Transformers, and if the experts are load-balanced, then each expert should expect on average T/n queries in a batch. In practice, the experts cannot expect perfect load balancing: in some batches, one expert might be underworked, while in other batches, it would be overworked. Since the inputs cannot move through the layer until every expert in the layer has finished the queries it is assigned, load balancing is important. The capacity factor is sometimes used to enforce a hard constraint on load balancing. Each expert is only allowed to process up to $c \cdot T/n$ queries in a batch. The ST-MoE report found $c \in [1.25, 2]$ to work well in practice.

Section 5

Applications to transformer models MoE layers are used in the largest transformer models, for which learning and

inferring over the full model is too costly. They are typically sparsely-gated, with sparsity 1 or 2. In Transformer models, the MoE layers are often used to select the feedforward layers (typically a linear-ReLU-linear network), appearing in each Transformer block after the multiheaded attention. This is because the feedforward layers take up an increasing portion of the computing cost as models grow larger. For example, in the Palm-540B model, 90% of parameters are in its feedforward layers. A trained Transformer can be converted to a MoE by duplicating its feedforward layers, with randomly initialized gating, then trained further. This is a technique called "sparse upcycling". There are a large number of design choices involved in Transformer MoE that affect the training stability and final performance. The OLMoE report describes these in some detail. As of 2023, models large enough to use MoE tend to be large language models, where each expert has on the order of 10 billion parameters. Other than language models, Vision MoE is a Transformer model with MoE layers. They demonstrated it by training a model with 15 billion parameters. MoE Transformer has also been applied for diffusion models. A series of large language models from Google used MoE. GShard uses MoE with up to top-2 experts per layer. Specifically, the top-1 expert is always selected, and the top-2th expert is selected with probability proportional to that experts' weight according to the gating function. Later, GLaM demonstrated a language model with 1.2 trillion parameters, each MoE layer using top-2 out of 64 experts. Switch Transformers use top-1 in all MoE layers. The NLLB-200 by Meta AI is a machine translation model for 200 languages. Each MoE layer uses a hierarchical MoE with two levels. On the first level, the gating function chooses to use either a "shared" feedforward layer, or to use the experts. If using the experts, then another gating function computes the weights and chooses the top-2 experts. MoE large language models can be adapted for downstream tasks by instruction tuning. In December 2023, Mistral AI released Mixtral 8x7B under Apache 2.0 license. It is a MoE language model with 46.7B parameters, 8 experts, and sparsity 2. They also released a version finetuned for instruction following. In March 2024, Databricks released DBRX. It is a MoE language model with 132B parameters, 16 experts, and sparsity 4. They also released a version finetuned for instruction following.